



FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA

(FIAP)

**CURSO SUPERIOR DE TECNOLOGIA EM INTELIGÊNCIA
ARTIFICIAL**

GRUPO SÃO PAULO E INTERIOR

JONAS TADEU V. FERNANDES

LEVI PASSOS SILVEIRA MARQUES

RAPHAEL DA SILVA

RAPHAEL DINELLI NETO

YAN PIMENTAL COTTA

CardiolA:

**IoT Assistente Cardiológico Conversacional com IBM Watsonx
Assistant e Flask**

SÃO PAULO

2026



**FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA
(FIAP)**

**CURSO SUPERIOR DE TECNOLOGIA EM INTELIGÊNCIA
ARTIFICIAL**

**GRUPO SÃO PAULO E INTERIOR
JONAS TADEU V. FERNANDES
LEVI PASSOS SILVEIRA MARQUES
RAPHAEL DA SILVA
RAPHAEL DINELLI NETO
YAN PIMENTAL COTTA**

CardiolA:

**IoT Assistente Cardiológico Conversacional com IBM Watsonx
Assistant e Flask**

Trabalho acadêmico apresentado ao Curso Superior de Tecnologia em Inteligência Artificial da Faculdade de Informática e Administração Paulista (FIAP), como requisito para obtenção de nota na disciplina Processamento de Linguagem Natural, Chatbots & Virtual Agents.

Tutor: Caique Nonato da Silva Bezerra

SÃO PAULO

2026



SUMÁRIO

1. INTRODUÇÃO	9
2 DESENVOLVIMENTO 1 – MODELAGEM DO ASSISTENTE CONVERSACIONAL.....	11
2.1 Definição do problema e escolha da plataforma	11
2.2 Estrutura conversacional	12
2.3 Entidades utilizadas	14
2.4 Fluxo de diálogo.....	15
2.5 Estrutura das perguntas.....	16
2.6 Classificação inicial das situações.....	17
2.7 Tratamento de exceções.....	18
3 DESENVOLVIMENTO 2 – INTEGRAÇÃO DO ASSISTENTE COM A APLICAÇÃO.....	19
3.1 Arquitetura da integração.....	19
3.2 Implementação do backend Flask	20
3.3 Controle de sessão	21
3.4 Comunicação com o Watson Assistant	21
3.5 Configuração do serviço IBM.....	22
3.6 Interface web.....	23
3.7 Comunicação entre JavaScript e Flask	24
3.8 Utilização de CORS	25
3.9 Endpoint de verificação.....	26
3.10 Fluxo completo da aplicação	27
3.11 Testes realizados.....	27
5 CONCLUSÃO.....	31



LISTA DE FIGURAS

Figura 1 – Arquitetura geral do Cardiola Fonte: Elaborado pelo autor (2026)..	10
Figura 2 – Ambiente utilizado para desenvolvimento do assistente	12
Figura 3 – Exemplos de treinamento da intenção de dor no peito	14
Figura 5 – Organização dos arquivos do projeto parte 1	19
Figura 6 – Organização dos arquivos do projeto parte 2.....	20
Figura 7 – Separação do Projeto Fonte: Elaborado pelo autor (2026).....	22
Figura 9 – Teste de interação com o Cardiola.....	29



LISTA DE TABELAS

Tabela 1 – Intenções utilizadas pelo CardiolA	13
Tabela 2 – Principais entidades utilizadas pelo assistente	14
Tabela 3 – Tecnologias utilizadas no projeto Fonte: Elaborado pelo autor (2026).	30



LISTA DE CÓDIGO

Código 1 – Recebimento da mensagem.....	20
Código 2 – Criação da sessão	21
Código 3 – Envio da mensagem	22
Código 4 – Leitura das configurações	23
Código 5 – Requisição ao backend	25
Código 6 – Configuração do CORS	26
Código 7 – Health check	26



RESUMO

Este trabalho apresenta o desenvolvimento do Cardiola, um assistente conversacional acadêmico voltado à interação inicial sobre sintomas cardiovasculares. A solução utiliza o IBM watsonx Assistant para processamento de linguagem natural, com modelagem de intenções, entidades, nós de diálogo, slots e regras condicionais responsáveis pela condução contextual da conversa. Para integração do serviço conversacional com aplicações externas, foi desenvolvido um backend em Python utilizando Flask, responsável pelo gerenciamento das requisições, sessões e comunicação com a API do Watson. Também foi implementada uma interface web utilizando HTML, CSS e JavaScript, permitindo o envio de mensagens e a apresentação das respostas em tempo real. O desenvolvimento possibilitou aplicar conceitos de NLP, APIs, arquitetura cliente-servidor, gerenciamento de contexto e integração entre frontend, backend e serviços de inteligência artificial. O Cardiola possui finalidade exclusivamente acadêmica, não realizando diagnósticos ou substituindo avaliação médica profissional, mas demonstrando possibilidades de aplicação de assistentes inteligentes em sistemas digitais de saúde.

Palavras-chave: Inteligência Artificial. Processamento de Linguagem Natural. Assistente Conversacional. IBM watsonx Assistant. Flask.



ABSTRACT

This paper presents the development of Cardiola, an academic conversational assistant designed for initial interaction regarding cardiovascular symptoms. The solution uses IBM watsonx Assistant for natural language processing, including the modeling of intents, entities, dialog nodes, slots, and conditional rules responsible for conducting contextual conversations. To integrate the conversational service with external applications, a backend was developed in Python using Flask, responsible for managing requests, sessions, and communication with the Watson API. A web interface was also implemented using HTML, CSS, and JavaScript, allowing users to send messages and receive responses in real time. The development process enabled the practical application of NLP, APIs, client-server architecture, context management, and integration between frontend, backend, and artificial intelligence services. Cardiola was developed exclusively for academic purposes and does not provide medical diagnoses or replace professional medical evaluation, while demonstrating potential applications of intelligent conversational assistants in digital healthcare systems.

Keywords: Artificial Intelligence. Natural Language Processing. Conversational Assistant. IBM watsonx Assistant. Flask.



1. INTRODUÇÃO

O avanço das tecnologias de Inteligência Artificial tem permitido a criação de sistemas capazes de interagir com usuários por meio de linguagem natural, tornando o acesso à informação mais simples, rápido e intuitivo. Entre essas tecnologias estão os assistentes conversacionais, também conhecidos como chatbots, que utilizam técnicas de Processamento de Linguagem Natural — Natural Language Processing (NLP) — para interpretar mensagens, identificar intenções e fornecer respostas contextualizadas.

No contexto da saúde, esses sistemas podem contribuir principalmente na organização de informações, orientação inicial e direcionamento do usuário. Entretanto, aplicações desse tipo devem possuir limitações bem definidas, uma vez que não substituem avaliação clínica, diagnóstico médico ou atendimento profissional.

Este projeto teve como objetivo desenvolver um assistente cardiológico conversacional, denominado CardiolA, utilizando a plataforma IBM watsonx Assistant, integrada a uma aplicação backend desenvolvida em Python com o framework Flask. O sistema foi projetado para reconhecer sintomas relacionados ao contexto cardiovascular, conduzir um fluxo inicial de perguntas e apresentar orientações de acordo com as informações fornecidas pelo usuário.

A solução foi dividida em duas partes principais. A primeira compreende a modelagem do assistente conversacional, criação das intenções, entidades, fluxos de diálogo e integração com o IBM watsonx Assistant. A segunda corresponde ao desenvolvimento de uma interface web capaz de consumir a API desenvolvida em Flask, permitindo que o usuário envie mensagens e visualize as respostas fornecidas pelo assistente.

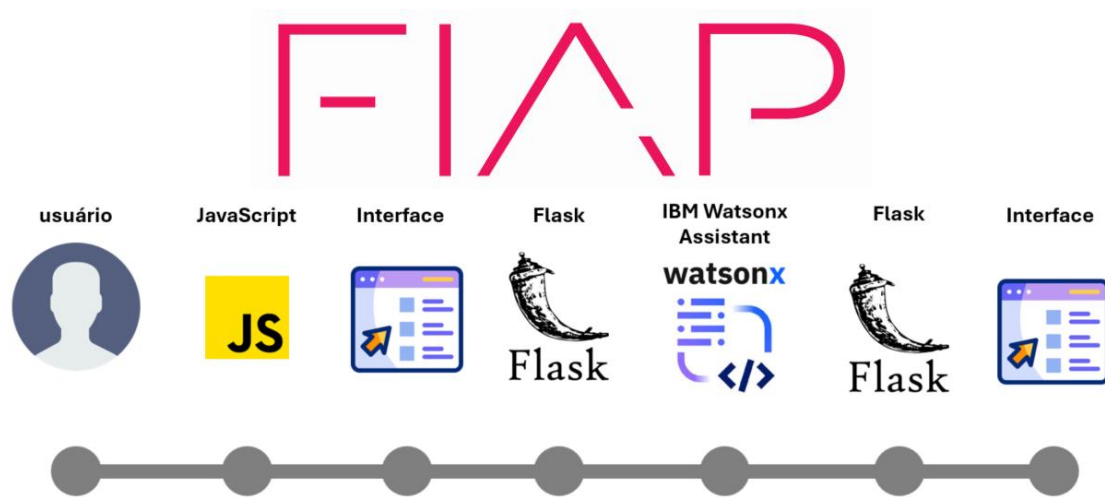


Figura 1 – Arquitetura geral do CardiolA

Fonte: Elaborado pelo autor (2026).

Segundo a documentação da IBM, o Watson Assistant permite construir aplicações conversacionais capazes de interpretar mensagens dos usuários por meio de intenções, entidades e estruturas de diálogo, possibilitando a criação de experiências conversacionais contextualizadas (IBM, s.d.).



2 DESENVOLVIMENTO 1 – MODELAGEM DO ASSISTENTE CONVERSACIONAL

2.1 Definição do problema e escolha da plataforma

A proposta inicial consistiu na criação de um assistente voltado ao atendimento inicial no contexto cardiológico. Para isso, foi utilizada a plataforma IBM watsonx Assistant, conforme os conceitos apresentados durante a disciplina.

A escolha da ferramenta ocorreu principalmente por sua capacidade de trabalhar com elementos essenciais de NLP, como:

- identificação de intenções;
- reconhecimento de entidades;
- controle do contexto da conversa;
- criação de nós de diálogo;
- integração com aplicações externas por meio de API.

Durante o processo de desenvolvimento ocorreram algumas dificuldades relacionadas ao acesso e configuração do ambiente IBM. Em determinado momento, não foi possível prosseguir com todas as configurações utilizando a conta inicialmente disponível, sendo necessário utilizar temporariamente uma conta de um colega que possuía acesso adequado ao ambiente necessário para o desenvolvimento acadêmico.

Essa etapa demandou atenção adicional, pois diferentes versões e configurações do watsonx Assistant apresentam alterações na forma de criação, publicação e integração do assistente.

Após a disponibilização correta do ambiente, optou-se pela utilização da estrutura baseada em Dialog, pois ela permitia representar de forma clara o fluxo de perguntas e respostas necessário para o projeto.

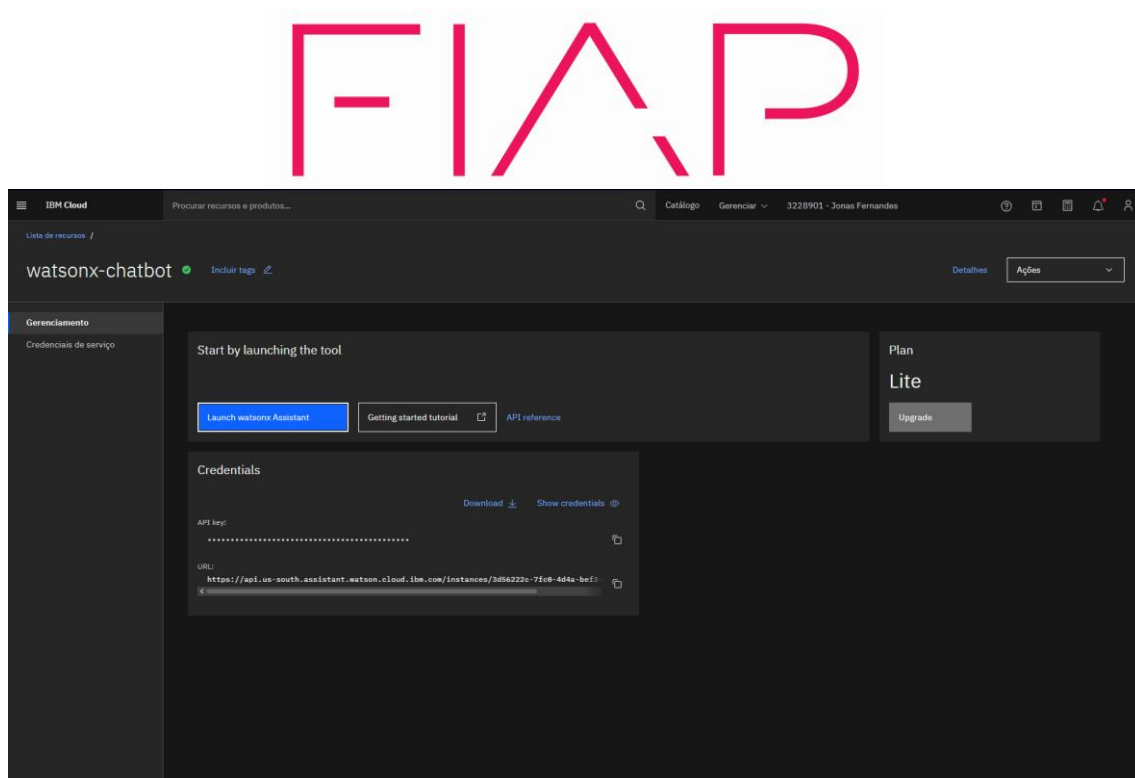


Figura 2 – Ambiente utilizado para desenvolvimento do assistente

Fonte: Elaborado pelo autor (2026).

2.2 Estrutura conversacional

A estrutura do Cardiola foi baseada na identificação dos principais sintomas relacionados a situações cardiovasculares que poderiam ser utilizados como ponto de entrada da conversa.

Foram definidas oito intenções principais:



Intenção	Objetivo
#chest_pain	Identificar relatos de dor no peito
#shortness_of_breath	Identificar falta de ar
#palpitations	Identificar palpitações
#dizziness	Identificar tontura
#fainting	Identificar episódios de desmaio
#cold_sweat	Identificar suor frio
#nausea	Identificar náusea
#swelling	Identificar inchaço

Tabela 1 – Intenções utilizadas pelo Cardiola

Fonte: Elaborado pelo autor (2026).

Cada intenção recebeu diferentes exemplos de frases para treinamento. Dessa maneira, o sistema não depende de uma única frase exata para reconhecer determinado sintoma.

Por exemplo, para a intenção relacionada à dor no peito, podem ser utilizadas expressões semelhantes a:

“I have chest pain.”

“My chest hurts.”

“I am feeling pain in my chest.”

“I have pressure in my chest.”

A utilização de diversos exemplos permite que o modelo consiga generalizar diferentes formas de manifestação de uma mesma intenção.

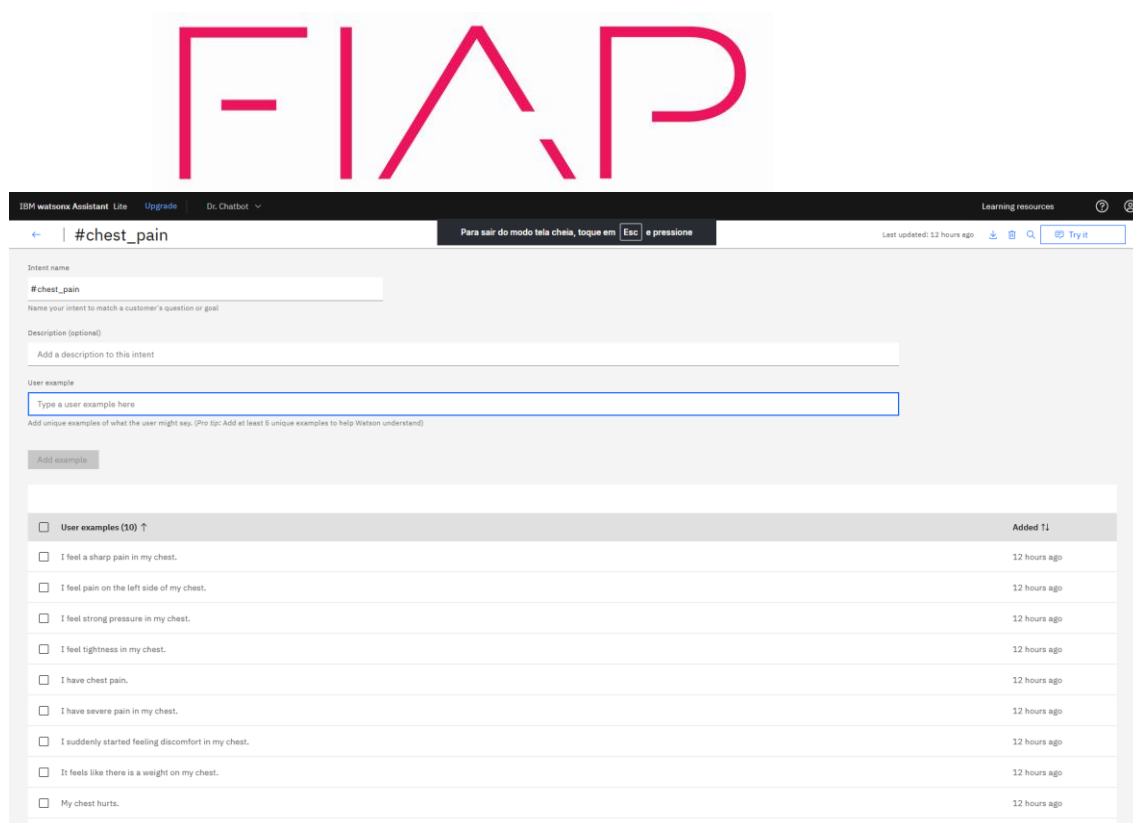


Figura 3 – Exemplos de treinamento da intenção de dor no peito

Fonte: Elaborado pelo autor (2026).

2.3 Entidades utilizadas

Além das intenções, foram criadas entidades responsáveis por extrair informações específicas das mensagens e respostas do usuário.

As principais entidades utilizadas foram:

Entidade	Função
@symptom	Identificação de sintomas
@severity	Grau de intensidade
@duration	Duração do sintoma
@pain_type	Tipo ou característica da dor
@yes_no	Respostas afirmativas ou negativas
@sys-number	Identificação de valores numéricos

Tabela 2 – Principais entidades utilizadas pelo assistente

Fonte: Elaborado pelo autor (2026).



A entidade de severidade, por exemplo, permite que o sistema interprete uma resposta numérica fornecida pelo usuário após uma pergunta relacionada à intensidade do sintoma.

Essa combinação entre intenção e entidades permite que uma frase seja decomposta em diferentes informações.

Por exemplo:

“I have severe chest pain for twenty minutes.”

Pode ser interpretada conceitualmente como:

```
Intent:
#chest_pain

Entities:
symptom = chest pain
severity = severe
duration = twenty minutes
```

O reconhecimento dessas informações permite tornar o fluxo conversacional mais estruturado.

2.4 Fluxo de diálogo

Após a identificação das intenções e entidades, foram desenvolvidos os nós de diálogo responsáveis por conduzir a conversa.

Quando uma intenção é reconhecida, o assistente inicia uma sequência de perguntas para coletar informações adicionais.

No caso de dor no peito, por exemplo, podem ser consideradas informações como:

- intensidade da dor;
- duração do sintoma;
- característica da dor;
- presença de outros sintomas;
- permanência do sintoma naquele momento.

O fluxo pode ser representado de maneira simplificada da seguinte forma:

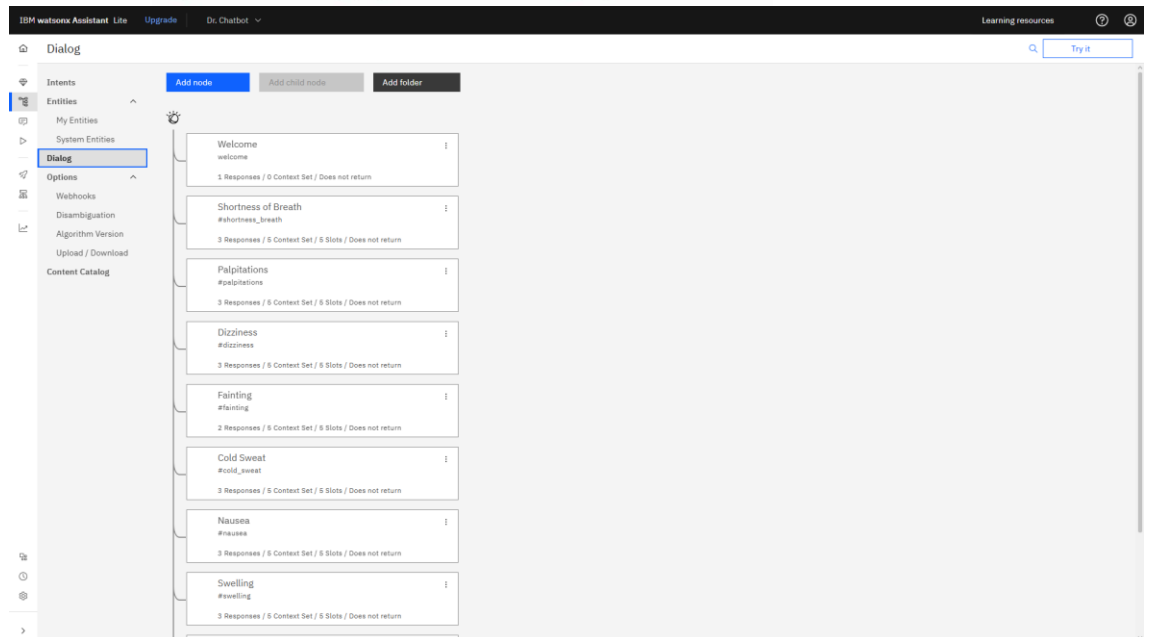


Figura 4 – Estrutura de diálogo criada no IBM watsonx Assistant

Fonte: Elaborado pelo autor (2026).

2.5 Estrutura das perguntas

As perguntas foram desenvolvidas em inglês, pois o conjunto de treinamento do assistente e seus fluxos foram estruturados utilizando esse idioma.

Um exemplo de interação pode ocorrer da seguinte maneira:

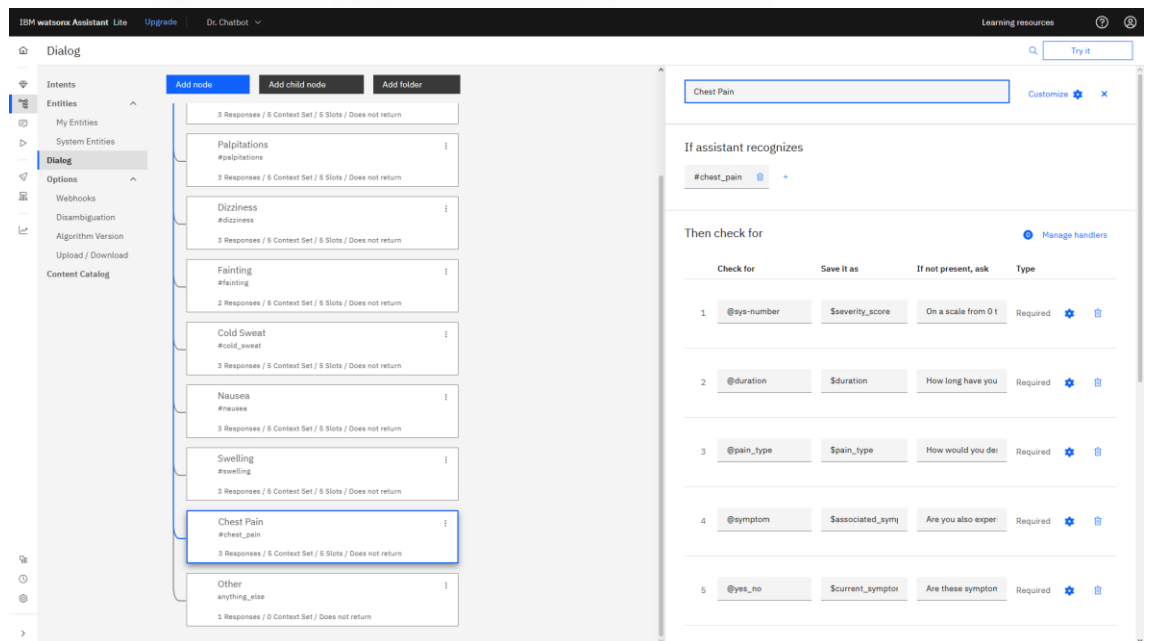


Figura 4 – Estrutura de diálogo criada no IBM watsonx Assistant

Fonte: Elaborado pelo autor (2026).

As respostas fornecidas em cada etapa são utilizadas para continuidade do fluxo.

Esse mecanismo demonstra a utilização de contexto conversacional, pois a interpretação de cada resposta depende da etapa anterior da conversa.

2.6 Classificação inicial das situações

Para fins acadêmicos, o projeto utiliza uma lógica simplificada de classificação da situação.

As respostas podem conduzir o fluxo para três níveis conceituais:

1. orientação geral;
2. recomendação de avaliação médica;
3. recomendação de atendimento emergencial.

É importante destacar que essa classificação não constitui protocolo médico validado e não realiza diagnóstico clínico.

Seu objetivo é demonstrar a aplicação de lógica condicional dentro de um sistema conversacional.



Em situações consideradas potencialmente graves pelo fluxo desenvolvido, o sistema orienta o usuário a procurar atendimento de emergência.

No contexto brasileiro, o sistema também apresenta a orientação de contato com o SAMU pelo número 192.

2.7 Tratamento de exceções

Outro ponto considerado durante o desenvolvimento foi o tratamento básico de entradas não reconhecidas.

Um assistente conversacional não deve considerar que todas as mensagens seguirão exatamente o fluxo esperado.

Assim, foram consideradas situações como:

- mensagens fora do contexto esperado;
- entradas sem intenção reconhecida;
- respostas diferentes das opções esperadas;
- valores numéricos;
- respostas afirmativas ou negativas.

Esse tratamento permite reduzir a possibilidade de interrupção completa do diálogo diante de uma mensagem inesperada.

O sistema também restringe sua atuação ao caráter educacional e informativo, evitando apresentar diagnóstico médico ou prescrição.



3 DESENVOLVIMENTO 2 – INTEGRAÇÃO DO ASSISTENTE COM A APLICAÇÃO

3.1 Arquitetura da integração

Após a construção do fluxo dentro do IBM watsonx Assistant, foi desenvolvida a integração com uma aplicação externa.

O backend foi implementado utilizando:

- Python;
- Flask;
- IBM Watson SDK;
- python-dotenv;
- Flask-CORS.

A aplicação possui a função de receber uma mensagem da interface, encaminhá-la para o IBM watsonx Assistant e retornar a resposta ao navegador.

A separação entre backend e serviço Watson foi escolhida para melhorar a organização do projeto.

Uma estrutura simplificada pode ser apresentada da seguinte forma:

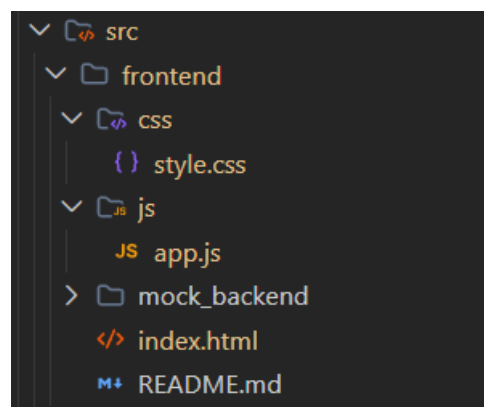


Figura 5 – Organização dos arquivos do projeto parte 1

Fonte: Elaborado pelo autor (2026).

Já a interface principal da Parte 2 foi mantida separadamente:

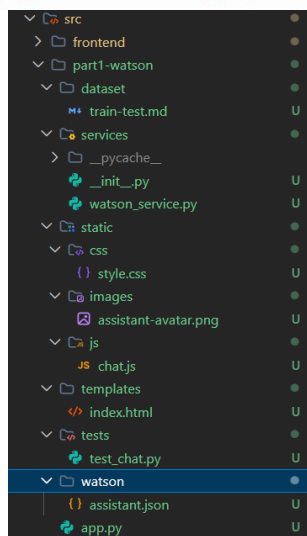


Figura 6 – Organização dos arquivos do projeto parte 2

Fonte: Elaborado pelo autor (2026).

Essa divisão permite separar a responsabilidade visual da lógica de integração.

3.2 Implementação do backend Flask

O arquivo app.py representa a principal camada HTTP da aplicação.

Sua função é disponibilizar endpoints que podem ser utilizados pela interface.

O endpoint principal é: POST /chat

Esse endpoint recebe um objeto JSON contendo a mensagem do usuário.

Código 1 – Recebimento da mensagem

```
data = request.get_json(silent=True) or {}  
message = data.get("message", "").strip()  
  
if not message:  
    return jsonify({  
        "error": "A message is required."  
    }), 400
```

Fonte: Elaborado pelo autor (2026).



O trecho apresentado no Código 1 verifica o corpo da requisição e impede que uma mensagem vazia seja encaminhada para o Watson.

Isso representa uma forma simples de validação de entrada.

3.3 Controle de sessão

Um aspecto importante do projeto é a manutenção do contexto entre diferentes mensagens.

Uma conversa com o Watson não pode ser tratada como mensagens totalmente independentes, pois o assistente precisa saber que uma resposta como: 8.

é relacionada à pergunta anterior sobre intensidade da dor.

Por esse motivo, o backend mantém um identificador de sessão.

Código 2 – Criação da sessão

```
if "user_id" not in session:
    session["user_id"] = str(uuid.uuid4())

if "watson_session_id" not in session:
    session["watson_session_id"] = create_session()
```

Fonte: Elaborado pelo autor (2026).

O identificador gerado com UUID permite distinguir diferentes usuários ou sessões de navegador sem armazenar diretamente informações pessoais.

Já o identificador `watson_session_id` é utilizado para preservar o contexto da conversa na comunicação com o serviço IBM.

3.4 Comunicação com o Watson Assistant

Após a criação da sessão, a mensagem é encaminhada para o serviço responsável pela comunicação com o Watson.



Código 3 – Envio da mensagem

```
response = send_message(  
    session["watson_session_id"],  
    message,  
    session["user_id"]  
)
```

Fonte: Elaborado pelo autor (2026).

Essa função encontra-se no arquivo: services/watson_service.py

A utilização de um arquivo separado foi adotada para evitar que o app.py concentre todas as responsabilidades.

Assim, o projeto apresenta uma separação entre:

Um diagrama de código em um fundo escuro que mostra a estrutura de arquivos do projeto. Cada arquivo é listado com uma seta e uma descrição de sua função.

```
app.py  
→ rotas HTTP e controle da aplicação  
  
watson_service.py  
→ comunicação com a IBM  
  
app.js  
→ comunicação entre frontend e backend  
  
HTML/CSS  
→ interface visual
```

Figura 7 – Separação do Projeto

Fonte: Elaborado pelo autor (2026).

Essa divisão contribui para manutenção e compreensão do projeto.

3.5 Configuração do serviço IBM

As credenciais utilizadas para comunicação com o serviço são obtidas por variáveis de ambiente.

Entre as configurações necessárias estão:

WATSON_API_KEY



WATSON_URL

WATSON_ASSISTANT_ID

WATSON_ENVIRONMENT_ID

Esses dados não devem permanecer diretamente no código-fonte público.

O arquivo .env é utilizado durante o desenvolvimento local, enquanto o .gitignore deve impedir sua publicação acidental no GitHub.

Essa prática contribui para evitar exposição de credenciais sensíveis.

Código 4 – Leitura das configurações

```
API_KEY = os.getenv("WATSON_API_KEY")
SERVICE_URL = os.getenv("WATSON_URL")
ASSISTANT_ID = os.getenv("WATSON_ASSISTANT_ID")
ENVIRONMENT_ID = os.getenv("WATSON_ENVIRONMENT_ID")
```

Fonte: Elaborado pelo autor (2026).

3.6 Interface web

Para a segunda etapa do projeto foi escolhida uma aplicação HTML, alternativa prevista no escopo da atividade.

A interface foi desenvolvida utilizando:

- HTML;
- CSS;
- JavaScript.

O HTML possui os principais elementos necessários para interação:

- cabeçalho;
- indicador de conexão;
- aviso de caráter educacional;
- área de mensagens;
- campo de texto;
- botão de envio.

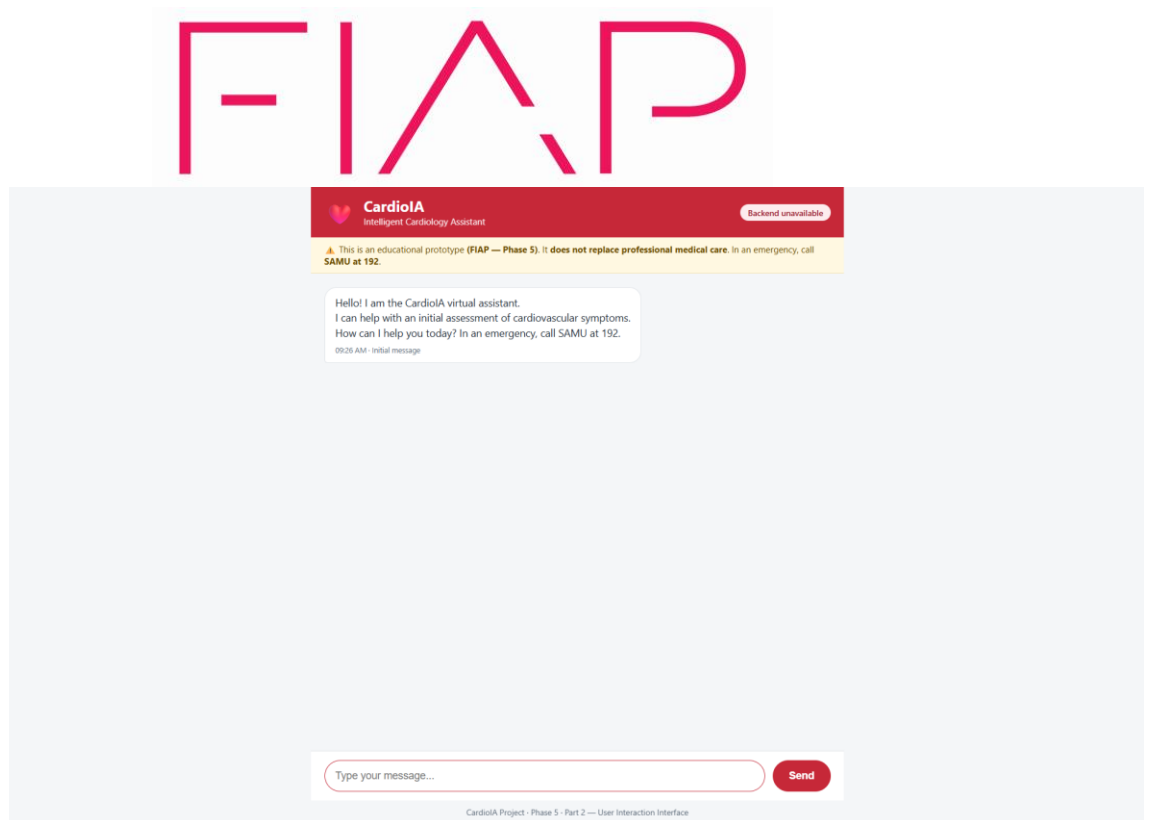


Figura 8 – Interface web utilizada para interação com o CardiolA

Fonte: Elaborado pelo autor (2026).

A escolha de HTML simplificou a implementação e permitiu concentrar o desenvolvimento nos conceitos principais da atividade: NLP, fluxo conversacional e integração com API.

3.7 Comunicação entre JavaScript e Flask

A comunicação entre frontend e backend ocorre utilizando a API Fetch do navegador.



Código 5 – Requisição ao backend

```
const response = await fetch(
  `${API_BASE}/chat`,
  {
    method: "POST",

    headers: {
      "Content-Type": "application/json"
    },

    body: JSON.stringify({
      message: text
    }),

    credentials: "include"
  }
);
```

Fonte: Elaborado pelo autor (2026).

O método utilizado é POST, uma vez que uma mensagem deve ser encaminhada ao servidor.

O objeto enviado possui a seguinte estrutura:

```
{
  "message": "I have chest pain"
}
```

O backend posteriormente retorna uma resposta semelhante a:

```
{
  "message": "I have chest pain",
  "response": "On a scale from 0 to 10, how severe is the pain?"
}
```

A resposta recebida é apresentada dinamicamente na área de chat.

3.8 Utilização de CORS

Como durante o desenvolvimento o backend Flask e a interface da Parte 2 são executados em portas diferentes, foi necessário habilitar comunicação entre origens distintas.

Por exemplo:

Frontend:



<http://127.0.0.1:5500>

Backend:

<http://127.0.0.1:5000>

Para essa situação foi utilizada a extensão Flask-CORS.

Código 6 – Configuração do CORS

```
CORS(  
    app,  
    resources={  
        r"/chat": {  
            "origins": [  
                "http://127.0.0.1:5500",  
                "http://localhost:5500"  
            ]  
        }  
    },  
    supports_credentials=True  
)
```

Fonte: Elaborado pelo autor (2026).

Essa configuração permite que o navegador realize requisições ao backend mesmo quando a interface está sendo executada por outro servidor local.

3.9 Endpoint de verificação

Também foi criado um endpoint destinado à verificação da disponibilidade do backend. GET /health

Código 7 – Health check

```
@app.route("/health", methods=["GET"])  
def health():  
    return jsonify({  
        "status": "ok",  
        "source": "watson"  
    })
```

Fonte: Elaborado pelo autor (2026).



A interface pode consultar esse endpoint para informar se a conexão está disponível.

Quando o backend está funcionando, a interface apresenta: Connected to Watson. Caso contrário: Backend unavailable.

Esse recurso melhora a experiência do usuário e auxilia também durante os testes.

3.10 Fluxo completo da aplicação

Após todas as etapas, o funcionamento final pode ser descrito pela seguinte sequência:

1. o usuário abre a interface;
2. o JavaScript verifica a disponibilidade do backend;
3. o usuário digita uma mensagem;
4. o JavaScript envia a mensagem por HTTP;
5. o Flask recebe a mensagem;
6. uma sessão é criada ou recuperada;
7. o backend encaminha a mensagem para o IBM watsonx Assistant;
8. o Watson interpreta a intenção;
9. o fluxo de diálogo determina a resposta;
10. o backend recebe a resposta;
11. a resposta é enviada ao frontend;
12. o JavaScript apresenta a mensagem no chat.

3.11 Testes realizados

Após a integração das diferentes partes do sistema, foram realizados testes utilizando os fluxos cadastrados no Watson.

Entre os sintomas avaliados estavam:

- chest pain;
- shortness of breath;



- palpitations;
- dizziness;
- fainting;
- cold sweat;
- nausea;
- swelling.

Inicialmente os testes foram realizados diretamente no ambiente do IBM watsonx Assistant.

Posteriormente, os mesmos fluxos foram testados através do backend Flask.

Por último, foi realizado o teste completo utilizando a interface da Parte 2.

O teste final confirmou o seguinte fluxo:

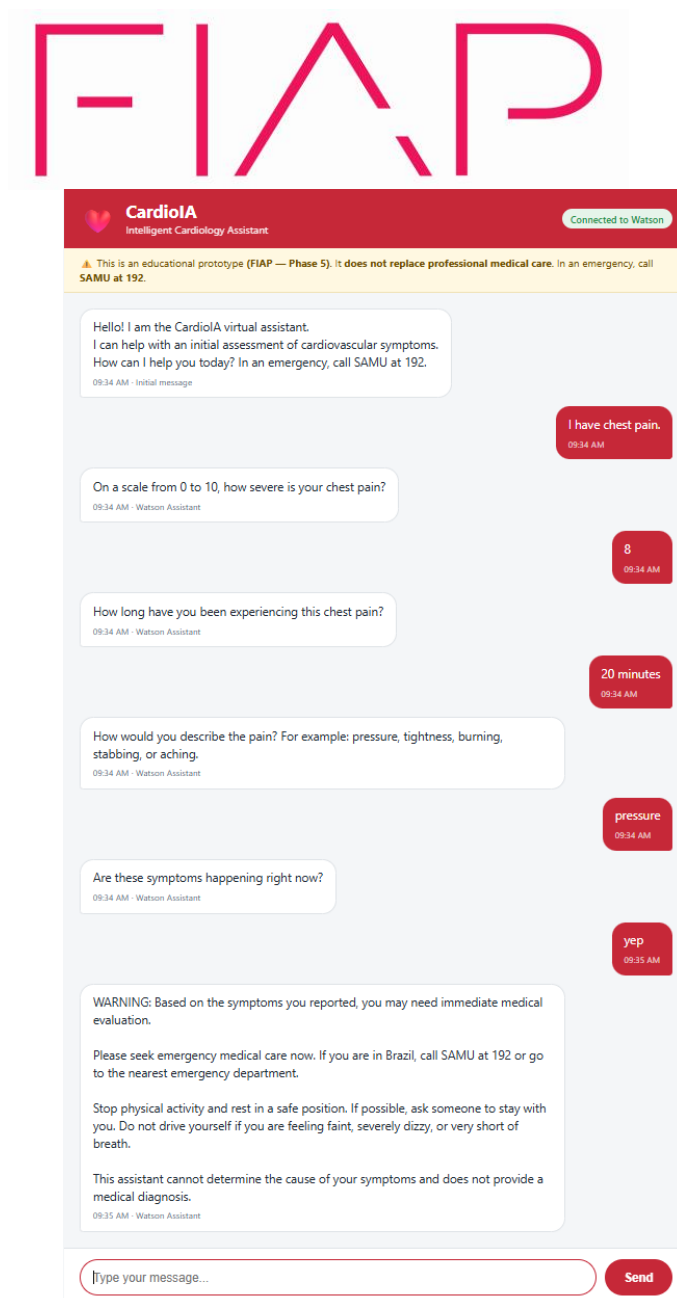


Figura 9 – Teste de interação com o CardiolA

Fonte: Elaborado pelo autor (2026).

Essa imagem é uma das mais importantes do relatório, porque demonstra que não se trata apenas de uma interface estática, mas de uma integração real entre frontend, backend e o serviço de NLP.

4 RESULTADOS OBTIDOS

Ao final do desenvolvimento, foi possível implementar uma aplicação funcional composta por diferentes tecnologias integradas.

A solução resultante possui:



Componente	Tecnologia
Processamento conversacional	IBM watsonx Assistant
NLP	Intents e Entities
Controle da conversa	Dialog / Slots
Backend	Python + Flask
Integração IBM	IBM Watson SDK
Interface	HTML + CSS
Comunicação frontend	JavaScript Fetch API
Controle de sessão	Flask Session
Comunicação entre origens	Flask-CORS
Configuração sensível	.env

Tabela 3 – Tecnologias utilizadas no projeto

Fonte: Elaborado pelo autor (2026).

A integração entre esses componentes demonstrou na prática como sistemas baseados em Inteligência Artificial podem ser incorporados a aplicações convencionais.



5 CONCLUSÃO

O desenvolvimento do CardiolA permitiu aplicar de maneira prática diferentes conceitos abordados durante a disciplina, principalmente aqueles relacionados a Processamento de Linguagem Natural, construção de assistentes conversacionais, APIs e integração entre sistemas.

A primeira etapa possibilitou compreender que um chatbot não é constituído apenas por respostas predefinidas. Para produzir uma conversa coerente, foi necessário estruturar intenções, entidades, exemplos de treinamento, variáveis, condições e diferentes caminhos dentro do diálogo.

O desenvolvimento dos fluxos também evidenciou a importância do contexto. Uma resposta simples como um número ou uma confirmação somente possui significado quando analisada considerando as perguntas anteriores.

Na segunda etapa, a integração do Watson com Flask demonstrou como um serviço de Inteligência Artificial pode ser disponibilizado para outras aplicações através de uma API. A separação entre interface, backend e camada de comunicação com o Watson tornou o projeto mais organizado e permitiu visualizar uma arquitetura semelhante à utilizada em aplicações reais.

Também foram observadas dificuldades durante o desenvolvimento, principalmente relacionadas à configuração do ambiente IBM, identificação correta dos parâmetros utilizados pela API, criação e preservação das sessões e integração entre diferentes aplicações executadas localmente. Essas dificuldades contribuíram para o aprendizado, pois exigiram análise de erros, testes de requisição e compreensão mais profunda sobre como os diferentes componentes se comunicam.

Do ponto de vista de mercado, tecnologias semelhantes podem ser utilizadas em diferentes cenários. Na área da saúde, por exemplo, assistentes virtuais podem auxiliar na coleta inicial de informações, organização de atendimento, direcionamento de usuários, esclarecimento de dúvidas e automatização de etapas administrativas.



Em hospitais, clínicas, plataformas de telemedicina e sistemas de atendimento, uma solução conversacional pode funcionar como uma camada inicial antes do contato com um profissional.

Entretanto, principalmente no contexto médico, esses sistemas devem ser utilizados como ferramentas de apoio e não como substitutos para profissionais da saúde. Questões relacionadas à segurança, privacidade, validação clínica, proteção de dados e responsabilidade devem ser consideradas antes da implantação de soluções dessa natureza em ambientes reais.

O projeto CardiolA atingiu o objetivo acadêmico de demonstrar a construção e integração de um assistente conversacional utilizando IBM watsonx Assistant, Python, Flask e uma interface web. Além do resultado técnico, o desenvolvimento proporcionou experiência prática sobre arquitetura de software, APIs, NLP, controle de estado, integração frontend/backend e utilização de serviços de Inteligência Artificial.